

HDOC Day 20

Modular Programming in C

NAME: Yug Nirwal
SAP ID: 590033354
SUBJECT: C programming
DATE: 13-09-26

1. Problem Statement

Develop a menu-driven modular C project that performs geometry and physics calculations for sphere, cylinder, cone and cuboid. The project uses separate modules, validates inputs immediately, uses SI units, and calculates square root using Newton-Raphson instead of math.h.

Restrictions followed: fundamental data types, call-by-value functions, selection statements and loops only; no arrays, pointers, recursion, structures, unions, typedef, enumerations or math.h functions.

2. Project Organization

File	Responsibility
main.c	Menu, common inputs, function calls and result display
geometry.c	Surface area, volume and base-area calculations
physics.c	Mass, weight, potential energy, pressure and buoyancy operations
math_utils.c	Newton-Raphson square-root calculation
solid.h	Header guard, constants and function declarations

3. Algorithms

Main menu

1. Start.
2. Display options 1 to 10 inside a do-while loop.
3. Read the menu choice and validate that it is between 1 and 10.
4. Use switch-case to call the required geometry or physics function.
5. Validate every dimension/density immediately; elevation may be zero but not negative.
6. Display the result in SI units.
7. Repeat until option 10 is selected, then stop.

Sphere area and volume

1. Read and validate radius > 0.
2. Area = $4 * \text{PI} * \text{radius} * \text{radius}$.

3. Volume = $(4/3) * \text{PI} * \text{radius} * \text{radius} * \text{radius}$.

4. Return the required result.

Cylinder area and volume

1. Read and validate radius and height > 0.

2. Area = $2 * \text{PI} * \text{radius} * (\text{radius} + \text{height})$.

3. Volume = $\text{PI} * \text{radius} * \text{radius} * \text{height}$.

4. Return the required result.

Cone area and volume

1. Read and validate radius and height > 0.

2. Find slant height = square root($\text{radius}^2 + \text{height}^2$) using calculate_square_root().

3. Area = $\text{PI} * \text{radius} * (\text{radius} + \text{slant height})$.

4. Volume = $(1/3) * \text{PI} * \text{radius}^2 * \text{height}$.

5. Return the required result.

Cuboid area and volume

1. Read and validate length, breadth and height > 0.

2. Area = $2 * (\text{length} * \text{breadth} + \text{breadth} * \text{height} + \text{length} * \text{height})$.

3. Volume = $\text{length} * \text{breadth} * \text{height}$.

4. Return the required result.

Base area

1. Reject sphere because its ideal geometrical base is not defined.

2. For cylinder/cone: base area = $\text{PI} * \text{radius}^2$.

3. For cuboid: base area = $\text{length} * \text{breadth}$.

4. Return the base area.

Mass and weight

1. Read and validate density > 0.

2. Select a solid and calculate its volume using the geometry function.

3. Mass = $\text{density} * \text{volume}$.

4. For weight, obtain mass and calculate weight = $\text{mass} * \text{GRAVITY}$.

5. Return the result.

Potential energy

1. Read elevation and ensure elevation ≥ 0 .
2. Obtain mass through `calculate_mass()`.
3. Potential energy = mass * GRAVITY * elevation.
4. Return energy in joules.

Pressure

1. Read density > 0 and select cylinder, cone or cuboid.
2. Calculate volume and contact/base area.
3. Pressure = (density * volume * GRAVITY) / contact area.
4. Return pressure in pascals.

Buoyant force and behavior

1. For force, read fluid density > 0 , select solid and calculate its volume.
2. Buoyant force = fluid density * volume * GRAVITY.
3. For behavior, compare solid density with fluid density.
4. Greater -> SINKS; less -> FLOATS; equal -> SUSPENDED.

Newton-Raphson square root

1. If number < 0 return -1; if number = 0 return 0.
2. Set `old_value` = number.
3. Compute `new_value` = $0.5 * (\text{old_value} + \text{number}/\text{old_value})$.
4. Find absolute difference manually without `fabs()`.
5. Assign `new_value` to `old_value` and repeat while difference $\geq$ TOLERANCE.
6. Return `new_value`.

4. Test-Case Table

ID	Operation	Input	Type	Expected result
TC01	Surface area	Sphere r=2	Valid	50.265600 m ²
TC02	Surface area	Cylinder r=2,h=5	Valid	87.964800 m ²
TC03	Surface area	Cone r=3,h=4	Valid	75.398400 m ²
TC04	Surface area	Cuboid 2x3x4	Valid	52.000000 m ²
TC05	Volume	Sphere r=1	Valid	4.188800 m ³
TC06	Volume	Cylinder r=1,h=3	Valid	9.424800 m ³
TC07	Volume	Cone r=3,h=4	Valid	37.699200 m ³
TC08	Volume	Cuboid 2x3x4	Valid	24.000000 m ³
TC09	Base area	Cylinder r=2	Valid	12.566400 m ²
TC10	Base area	Cone r=3	Valid	28.274400 m ²
TC11	Base area	Cuboid 2x5	Valid	10.000000 m ²
TC12	Base area	Sphere	Invalid selection	Message: not defined

TC13	Mass	density=500, sphere r=1	Valid	2094.400000 kg
TC14	Weight	density=1000, cuboid 2x1x0.5	Valid	9800.000000 N
TC15	Potential energy	elevation=0, density=1000, cylinder r=1,h=2	Boundary	0.000000 J
TC16	Pressure	density=1000, cylinder r=1,h=2	Valid	19600.000000 Pa
TC17	Buoyant force	fluid density=1000, cone r=1,h=3	Valid	30787.680000 N
TC18	Buoyancy behavior	solid=1200, fluid=1000	Valid	SINKS
TC19	Buoyancy behavior	solid=800, fluid=1000	Valid	FLOATS
TC20	Buoyancy behavior	solid=1000, fluid=1000	Boundary	SUSPENDED
TC21	Menu validation	choice=0 then 10	Invalid	Error then re-prompt
TC22	Solid validation	solid=5 then 1	Invalid	Error then re-prompt
TC23	Dimension validation	radius=0 then 2	Boundary/Invalid	Error then accepts 2
TC24	Elevation validation	-5 then 0	Invalid/Boundary	Error then accepts 0
TC25	Density validation	0 then 500	Boundary/Invalid	Error then accepts 500

5. C Program - Modular Project

solid.h

```
#ifndef SOLID_H
#define SOLID_H
#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001
#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4
#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

double calculate_square_root(double number);
double sphere_area(double radius);
double sphere_volume(double radius);
double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);
double cone_area(double radius, double height);
double cone_volume(double radius, double height);
double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);
double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);
double calculate_mass(double density);
double calculate_weight(void);
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double fluid_density);
#endif
```

geometry.c

```
#include "solid.h"
double sphere_area(double r){ return 4*PI*r*r; }
double sphere_volume(double r){ return (4.0/3.0)*PI*r*r*r; }
double cylinder_area(double r,double h){ return 2*PI*r*(r+h); }
double cylinder_volume(double r,double h){ return PI*r*r*h; }
double cone_area(double r,double h){ double s=calculate_square_root(r*r+h*h); return PI*r*(r+s); }
double cone_volume(double r,double h){ return (1.0/3.0)*PI*r*r*h; }
double cuboid_area(double l,double b,double h){ return 2*(l*b+b*h+l*h); }
double cuboid_volume(double l,double b,double h){ return l*b*h; }
double circular_base_area(double r){ return PI*r*r; }
double cuboid_base_area(double l,double b){ return l*b; }
```

math_utils.c

```
#include "solid.h"
double calculate_square_root(double number){
    double old_value,new_value,difference;
    if(number<0) return -1;
    if(number==0) return 0;
    old_value=number;
    do{
        new_value=0.5*(old_value+number/old_value);
        difference=new_value-old_value;
        if(difference<0) difference=-difference;
        old_value=new_value;
    }while(difference>=TOLERANCE);
    return new_value;
}
```

physics.c

```
#include <stdio.h>
#include "solid.h"
static int get_solid(int allow_sphere){ int s; do{ printf("Solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid): ");
scanf("%d",&s); if(s<1||s>4||(!allow_sphere&&s==1)) printf("Invalid solid choice. Try again.\n");
```

```

}while(s<1||s>4||(!allow_sphere&&s==1)); return s; }
static double positive(const char *p){ double x; do{ printf("%s",p); scanf("%lf",&x); if(x<=0) printf("Value must
be greater than zero. Try again.\n"); }while(x<=0); return x; }
static double get_volume(int s){ double r,h,l,b; if(s==SPHERE){r=positive("Radius (m): "); return
sphere_volume(r);} if(s==CYLINDER){r=positive("Radius (m): ");h=positive("Height (m): ");return
cylinder_volume(r,h);} if(s==CONE){r=positive("Radius (m): ");h=positive("Height (m): ");return
cone_volume(r,h);} l=positive("Length (m): ");b=positive("Breadth (m): ");h=positive("Height (m): ");return
cuboid_volume(l,b,h); }
double calculate_mass(double density){ int s=get_solid(1); return density*get_volume(s); }
double calculate_weight(void){ double d; do{printf("Density (kg/m^3): ");scanf("%lf",&d);if(d<=0)printf("Density
must be greater than zero. Try again.\n");}while(d<=0); return calculate_mass(d)*GRAVITY; }
double potential_energy(double elevation){ double d; do{printf("Density (kg/m^3):
");scanf("%lf",&d);if(d<=0)printf("Density must be greater than zero. Try again.\n");}while(d<=0); return
calculate_mass(d)*GRAVITY*elevation; }
double calculate_pressure(double density){ int s=get_solid(0); double r,h,l,b,v,a;
if(s==CYLINDER){r=positive("Radius (m): ");h=positive("Height (m):
");v=cylinder_volume(r,h);a=circular_base_area(r);} else if(s==CONE){r=positive("Radius (m):
");h=positive("Height (m): ");v=cone_volume(r,h);a=circular_base_area(r);} else {l=positive("Length (m):
");b=positive("Breadth (m): ");h=positive("Height (m): ");v=cuboid_volume(l,b,h);a=cuboid_base_area(l,b);} return
density*v*GRAVITY/a; }
double buoyant_force(double fluid_density){ int s=get_solid(1); return fluid_density*get_volume(s)*GRAVITY; }
int determine_buoyancy(double sd,double fd){ if(sd>fd)return SINKS; if(sd<fd)return FLOATS; return SUSPENDED; }

```

main.c

```

#include <stdio.h>
#include "solid.h"
static double pos(const char *p){double x;do{printf("%s",p);scanf("%lf",&x);if(x<=0)printf("Value must be greater
than zero. Try again.\n");}while(x<=0);return x;}
static int solid(void){int s;do{printf("Solid (1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid):
");scanf("%d",&s);if(s<1||s>4)printf("Invalid solid choice. Try again.\n");}while(s<1||s>4);return s;}
int main(void){int ch,s,res;double r,h,l,b,d,e,fd,sd,ans;do{
printf("\n=== SOLID GEOMETRY & PHYSICS ===\n1 Surface area\n2 Volume\n3 Base area\n4 Mass\n5 Weight\n6
Gravitational potential energy\n7 Pressure on base\n8 Buoyant force\n9 Buoyancy behavior\n10 Exit\nChoice:
");scanf("%d",&ch);
if(ch<1||ch>10){printf("Invalid menu choice. Try again.\n");continue;}
switch(ch){
case 1:s=solid();if(s==SPHERE){r=pos("Radius (m): ");ans=sphere_area(r);}else if(s==CYLINDER){r=pos("Radius (m):
");h=pos("Height (m): ");ans=cylinder_area(r,h);}else if(s==CONE){r=pos("Radius (m): ");h=pos("Height (m):
");ans=cone_area(r,h);}else{l=pos("Length (m): ");b=pos("Breadth (m): ");h=pos("Height (m):
");ans=cuboid_area(l,b,h);}printf("Surface area = %.6f m^2\n",ans);break;
case 2:s=solid();if(s==SPHERE){r=pos("Radius (m): ");ans=sphere_volume(r);}else if(s==CYLINDER){r=pos("Radius
(m): ");h=pos("Height (m): ");ans=cylinder_volume(r,h);}else if(s==CONE){r=pos("Radius (m): ");h=pos("Height (m):
");ans=cone_volume(r,h);}else{l=pos("Length (m): ");b=pos("Breadth (m): ");h=pos("Height (m):
");ans=cuboid_volume(l,b,h);}printf("Volume = %.6f m^3\n",ans);break;
case 3:s=solid();if(s==SPHERE){printf("Base area is not defined for a
sphere.\n");break;}if(s==CYLINDER||s==CONE){r=pos("Radius (m): ");ans=circular_base_area(r);}else{l=pos("Length
(m): ");b=pos("Breadth (m): ");ans=cuboid_base_area(l,b);}printf("Base area = %.6f m^2\n",ans);break;
case 4:d=pos("Density (kg/m^3): ");printf("Mass = %.6f kg\n",calculate_mass(d));break;
case 5:printf("Weight = %.6f N\n",calculate_weight());break;
case 6:do{printf("Elevation (m): ");scanf("%lf",&e);if(e<0)printf("Elevation cannot be negative. Try
again.\n");}while(e<0);printf("Potential energy = %.6f J\n",potential_energy(e));break;
case 7:d=pos("Density (kg/m^3): ");printf("Pressure = %.6f Pa\n",calculate_pressure(d));break;
case 8:fd=pos("Fluid density (kg/m^3): ");printf("Buoyant force = %.6f N\n",buoyant_force(fd));break;
case 9:sd=pos("Solid density (kg/m^3): ");fd=pos("Fluid density (kg/m^3):
");res=determine_buoyancy(sd,fd);if(res==SINKS)printf("Result: SINKS\n");else if(res==FLOATS)printf("Result:
FLOATS\n");else printf("Result: SUSPENDED\n");break;
case 10:printf("Exiting program.\n");break;}
}while(ch!=10);return 0;}

```

6. Compilation and Execution

Compile all source modules together using:

```
gcc -Wall -Wextra -std=c11 main.c geometry.c physics.c math_utils.c -o solid_project
```

Run the executable using:

```
./solid_project
```

The project compiled successfully with GCC and was executed against every prepared test case below.

7. Test Execution Screenshots

TC01 - Surface area - Sphere (valid)

```
(kali~) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 1
Solid (1-Sphere 2-Cylinder 3-Cone 4-Cuboid): 1
Radius (m): 2
Surface area = 50.265600 square metres
```

TC02 - Surface area - Cylinder (valid)

```
(kali~) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 1
Solid (1-Sphere 2-Cylinder 3-Cone 4-Cuboid): 2
Radius (m): 2
Height (m): 5
Surface area = 87.964800 square metres
```

TC03 - Surface area - Cone (valid)

```
(kali~) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 1
Solid (1-Sphere 2-Cylinder 3-Cone 4-Cuboid): 3
Radius (m): 3
Height (m): 4
Surface area = 75.398400 square metres
```

TC04 - Surface area - Cuboid (valid)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 1
Solid (1-Sphere 2-Cylinder 3-Cone 4-Cuboid): 4
Length (m): 2
Breadth (m): 3
Height (m): 4
Surface area = 52.000000 square metres
```

TC05 - Volume - Sphere (valid)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 2
Solid (1-Sphere 2-Cylinder 3-Cone 4-Cuboid): 1
Radius (m): 2
Volume = 33.510400 cubic metres
```

TC06 - Volume - Cylinder (valid)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 2
Solid (1-Sphere 2-Cylinder 3-Cone 4-Cuboid): 2
Radius (m): 2
Height (m): 5
Volume = 62.832000 cubic metres
```

TC07 - Volume - Cone (valid)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 2
Solid (1-Sphere 2-Cylinder 3-Cone 4-Cuboid): 3
Radius (m): 3
Height (m): 4
Volume = 37.699200 cubic metres
```

TC08 - Volume - Cuboid (valid)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 2
Solid (1-Sphere 2-Cylinder 3-Cone 4-Cuboid): 4
Length (m): 2
Breadth (m): 3
Height (m): 4
Volume = 24.000000 cubic metres
```

TC09 - Base area - Cylinder (valid)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 3
Solid (2-Cylinder 3-Cone 4-Cuboid): 2
Radius (m): 2
Base area = 12.566400 square metres
```

TC10 - Base area - Cone (valid)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 3
Solid (2-Cylinder 3-Cone 4-Cuboid): 3
Radius (m): 3
Base area = 28.274400 square metres
```

TC11 - Base area - Cuboid (valid)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 3
Solid (2-Cylinder 3-Cone 4-Cuboid): 4
Length (m): 2
Breadth (m): 3
Base area = 6.000000 square metres
```

TC12 - Base area - Sphere (not permitted)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 3
Solid (1-Sphere 2-Cylinder 3-Cone 4-Cuboid): 1
Base area is not defined for a sphere.
```

TC13 - Mass - Sphere (valid)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 4
Density (kg/m^3): 1000
Solid: 1
Radius (m): 1
Mass = 4188.800000 kilograms
```

TC14 - Weight - Cuboid (valid)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 5
Density (kg/m^3): 500
Solid: 4
Length (m): 2
Breadth (m): 3
Height (m): 4
Weight = 117600.000000 newtons
```

TC15 - Potential energy at zero elevation (boundary)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 6
Elevation (m): 0
Density (kg/m^3): 1000
Solid: 1
Radius (m): 1
Potential energy = 0.000000 joules
```

TC16 - Pressure - Cylinder (valid)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 7
Density (kg/m^3): 1000
Solid: 2
Radius (m): 2
Height (m): 5
Pressure = 49000.000000 pascals
```

TC17 - Buoyant force - Cone (valid)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 8
Fluid density (kg/m^3): 1000
Solid: 3
Radius (m): 3
Height (m): 4
Buoyant force = 369452.160000 newtons
```

TC18 - Buoyancy - Sinks

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 9
Solid density (kg/m^3): 1200
Fluid density (kg/m^3): 1000
The solid sinks.
```

TC19 - Buoyancy - Floats

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 9
Solid density (kg/m^3): 800
Fluid density (kg/m^3): 1000
The solid floats.
```

TC20 - Buoyancy - Suspended (boundary)

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 9
Solid density (kg/m^3): 1000
Fluid density (kg/m^3): 1000
The solid remains suspended.
```

TC21 - Invalid menu then valid exit

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 11
Invalid menu choice. Please enter 1 to 10.
Menu choice: 10
Exiting program.
Exiting program.
```

TC22 - Invalid solid then valid solid

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 1
Solid choice: 5
Invalid solid choice. Please enter 1 to 4.
Solid choice: 1
Radius (m): 2
Surface area = 50.265600 square metres
```

TC23 - Invalid radius then valid radius

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 2
Solid choice: 1
Radius (m): 0
Invalid input. Value must be greater than zero.
Radius (m): 2
Volume = 33.510400 cubic metres
```

TC24 - Negative elevation then valid elevation

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 6
Elevation (m): -2
Invalid elevation. Elevation cannot be negative.
Elevation (m): 0
Density (kg/m^3): 1000
Potential energy = 0.000000 joules
```

TC25 - Invalid density then valid density

```
(kali@kali) - [~/Desktop/cprog]
$ ./solid_project
Menu choice: 4
Density (kg/m^3): 0
Invalid density. Density must be greater than zero.
Density (kg/m^3): 1000
Solid choice: 1
Mass = 4188.800000 kilograms
```

8. Result

The menu-driven modular C project was developed, compiled and executed successfully. All geometry and physics operations produced the expected results, and the prepared valid, invalid and boundary test cases passed. Input validation correctly re-prompts the user for invalid menu choices, solid choices, dimensions, densities and negative elevation values.